

Natural Language Processing

“Voice Driven Quote Assistant”

(2024-2025)

Shri Prakash Yadav: D03000084, University Of Napels Federico II

Chapters

1. Abstract
2. Introduction
3. System Architecture
4. Quote Data Extraction & Preprocessing
5. Knowledge Graph
6. Retrieval Module
7. Language Understanding & Answering
8. Voice Interface
9. Integration & Orchestration
10. Evaluation
11. Conclusion & Future Work

1. Abstract:

I present voice driven quote assistant, a hands-free assistant that completes and attributes quotations using a strictly grounded retrieval pipeline. Spoken input is transcribed with ASR, a fragment/intent detector identifies whether to finish a quote or answer metadata queries, and a Neo4j graph database returns facts (quote text, author, source of the quote, and dispute/misattribution relations) through a full-text index. Responses are constrained to retrieved facts i.e. *finish the quote* emits quote text only, whereas *who/source/when/disputed* return deterministic facts from the active context. The system supports speaker registration and identification for per user text-to-speech (voice/rate/volume) preferences. I have described the architecture, graph schema, retrieval and ranking, grounded reasoning, and the voice layer in this report. I also tried to evaluate retrieval with precision and recall on the text index, and conducted a small human evaluation focusing on retrieval accuracy, latency, and TTS.

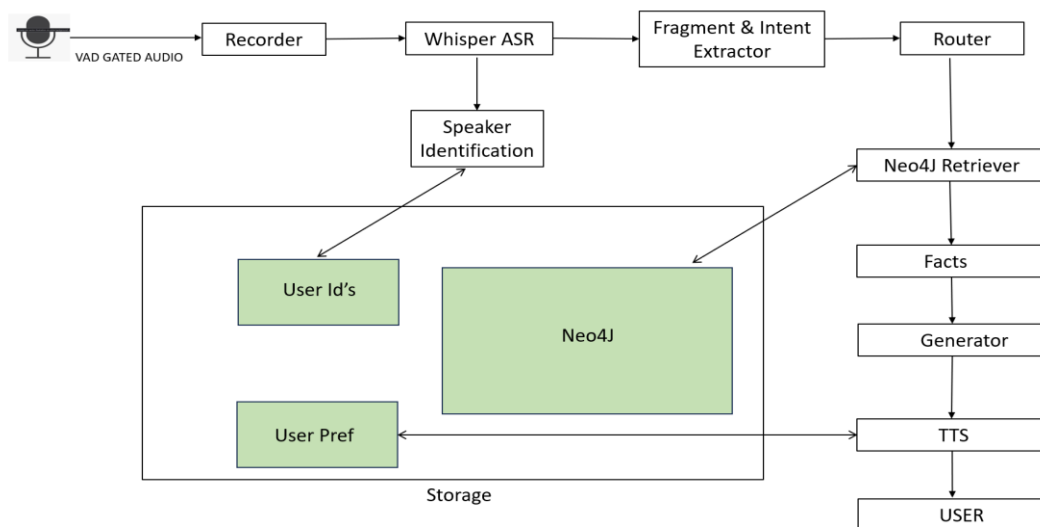
2. Introduction:

Most of us remember quotes in fragments, enough to feel the meaning, not enough to say them with confidence. Purely generative systems are very helpful, but they can also make things up. With this assistant, we take a different path. I treat quotes as facts that are saved in a graph database. When you ask it to finish a quote, it returns the exact text from the database. When you ask who said it, or whether it is disputed, it will pull those answers from recorded relationships. No guesswork or hallucination, only truth.

The voice assistant listens, transcribes, and then figures out the intent of the user. I kept the experience predictable by handling clear, deterministic commands first (reset, register, change voice/rate/volume), and I used the LLM for spotting the quote fragment inside messy, conversational speech and routing the request when the regex based rules don't cover it.

This assistant lets you register your voice, recognizes you later, and speaks back using your saved preferences for voice, speed, and volume.

3. System Architecture



- **Input Layer:**

- Microphone + VAD recorder captures speech segments.
- Whisper ASR transcribes audio into text.

- **Understanding Layer:**

- Fragment & Intent Extractor identifies quote fragments, metadata queries (who/source/disputed), or control commands (reset, register, change voice/rate/volume).

- **Router decides the next step:**
 - Control intents → handled directly.
 - Query intents → sent to Neo4j Retriever.
- **Retrieval Layer:**
 - Neo4j Retriever runs full-text queries on the quoteTextFT index.
 - Returns best-matching quotes node with properties (text, source, heading_context) and relationships (SAID_BY, ABOUT, MISATTRIBUTED_TO, DISPUTED_WITH).
- **Answer Generation Layer:**
 - Facts are strictly grounded in database output.
 - Generator formats deterministic responses:
 - *Finish quote* → only quote text.
 - *Who/source/disputed* → factual metadata.
- **Speaker Personalization Layer:**
 - Speaker Module manages registration, enrollment, and identification.
 - User preferences (voice, rate, volume) stored and applied.
- **Output Layer:**
 - TTS (pyttsx3) synthesizes responses with per-user settings.
 - Spoken output delivered back to the user.

4. Quote Dataset Extraction & Preprocessing:

4.1 Quote Dataset Extraction

The very first step in building my system was to extract the raw data from the Wiki quote dump. I realized that the individual quote pages were not written in a strict format that can be applied to whole dump. Each article page has its own style, different kinds of headings, and sometimes even multiple languages. Therefore, I decided to follow a medium-pattern approach, a set of rules that would work for most articles, even if not perfect for every single one.

The idea was that if a new line starting with *** (first order list under any heading or sub-heading) usually represents a **quote**, and a line starting with **** (*second order list*) usually represents the **source** of the quote. I set up the parser that way — *** meant “this is a quote,” and **** meant “this is the source for the last quote I saw.”

Ex:-

```
* ""Life is like riding a bicycle. To keep your balance you must keep moving.""
** Letter to his son Eduard (5 February 1930), as quoted in [[wikipedia:Walter Isaacson|Walter Isaacson]], "Einstein: His Life and Universe" (2007), p. 367

* ""I believe that whatever we do or live for has its causality; it is good, however, that we cannot see through to it.""
** Interview with [[Rabindranath Tagore]] (14 April 1930), published in "The Religion of Man" (1930) by Rabindranath Tagore, p. 222, and in "The Tagore Reader" (1971) edited by [[wikipedia:Amiya Chakravarty|Amiya Chakravarty]]
```

Along the way, I realized that the headings on each page often carried extra information. For example, in author pages there are headings like *1930s* or *On Music*. These were not part of the quotes themselves, but they seemed informative, so I extracted them as heading context.

Ex:-

```
==== Wisahart interview (1938) ====
: <small>M. K. Wisahart, "A Close Look at the World's Greatest Thinker", American Magazine, June 1938. Quotes from the interview appear on pp. 52-53 of [http://books.google.com/books?id=1_Gktr_D7F
* Every man knows that in his work he does best and accomplishes most when he has attained a proficiency that enables him to work intuitively. That is, there are things which we come to know so well that w

* I do not believe in a God who maliciously or arbitrarily interferes in the personal affairs of mankind. "My religion consists of a humble admiration for the vast power which manifests itself in that s

* "Much reading after a certain age diverts the mind from its creative pursuits. Any man who reads too much and uses his own brain too little falls into lazy habits of thinking," just as the man who sp

* I have only two rules which I regard as principles of conduct. The first is: Have no rules. The second is: Be independent of the opinion of others.

|
==== "Religion and Science" (1938) ====
[[File:Nebulabrot (5000, 500, 50).png|thumb|Everything that the [[human]] race has done and [[thought]] is concerned with the [[satisfaction]] of deeply felt [[needs]] and the assuagement of [[pain]].]]
: <small>Originally written for the "New York Times Magazine" (9 November 1938); a version with altered wording appeared in "Ideas and Opinions" (1954)</small>
[[File:San Francisco de Asis, por José de Ribera.jpg|thumb|We find precisely among the heretics of all ages men who were inspired by this highest [[religious]] experience; often they appeared to their cont
[[File:Cristo degli abissi.jpg|thumb|It is the most important function of [[art]] and [[science]] to awaken this [[feeling]] and keep it [[alive]] in those who are receptive to it.]]
[[File:Spirale_Ulam_150.jpg|thumb|The individual feels the vanity of human desires and aims, and the nobility and marvelous order which are revealed in nature and in the world of thought.]]
[[File:Orbitals7.png|thumb|A contemporary has rightly said that the only deeply [[religious]] [[people]] of our largely [[materialistic]] age are the earnest men of research.]]
```

Also, I refined the **author attribution rules**. If the page title was “John Cage”, I could assume he was the author of the quotes. But in special sections, I needed to override this:

- If the section heading was *Misattributed*, I marked the author as “Misattributed to John Cage.”
- If the section was *Disputed*, I changed it to “Disputed with John Cage.”
- If the section was *Quotes about John Cage*, then I left the author as unknown, because in those cases John Cage is the subject, not the speaker. Even if the source contains the original author of the quotes about John Cage, it is very hard to extract in an automated way, that why I left the author for about section as unknown.

Extracting the source of the quote was the tricky part. In many entries, the **last second-level list item** after a quote turned out to be the most informative. Often it carried page numbers, chapter references, or other citation details which, combined with the heading context, gave a fairly precise source. That said, when multiple second-level items followed a quote, the true source could have been any of them. Choosing the last one was a trade-off: most of the time it worked well, but not always. Sometimes those lines contained translations or vague notes instead of a proper citation, so the “source” field wasn’t perfectly clean. Still, treating the last second-level item as the source gave me a workable compromise across most of the data samples.

Ex:-

```
* "'The mass of a body is a measure of its energy content.'"
** "Ist die Trägheit eines Körpers von seinem Energieinhalt abhängig?" (&quot;Does the inertia of a body depend upon its energy content?&quot;)
```

After all these adjustments, I was able to extract about **30,000 records** from the dump. I kept the sample small for better manageability, which can be easily extended later. Each extracted record contained a quote, source, author, heading_context.

This raw dataset still had noise and redundancies, but it was a solid base. I cleaned and pre-processed this to make it suitable for the graph database.

4.2 EDA & Preprocessing

I performed some **Exploratory Data Analysis (EDA)** to understand the dataset and what kinds of problems were present.

The EDA report is below.

4.2.1 Exploratory Data Analysis (EDA)

- Dataset shape: The extracted file contained **30,000 records** with 4 features: quote, source, author, and heading_context. Out of these, only author was fully populated. quote had 29,984 non-null values (so 16 were missing), source had 22,914 non-null values, and heading_context had 20,671 non-null values.

- Null quotes – Chinese proverbs issue: On inspecting missing quotes, mostly came from the **Chinese proverbs** page.
 - Count: **16 records** with NaN quotes and 45 with just punctuation.
 - Cause: The extractor focused on characters only, but the Chinese characters were kept as hyperlinks. So, the extractor skipped this. It just extracted the punctuation if present, else NaN. And the translation of the quote along with the source ended in the source column.

Ex:-

- [不聞不若聞之，聞之不若見之，見之不若知之，知之不若行之；學至於行之而止矣。](#)
 - Transliteration (pinyin): *Bù wén bù ruò wén zhī, wén zhī bù ruò jiàn zhī, jiàn zhī bù ruò zhī zhī, zhī zhī bù ruò xíng zhī; xué zhī yú xíng zhī ér zhǐ yǐ.*
 - Traditional: 不聞不若聞之，聞之不若見之，見之不若知之，知之不若行之；學至於行之而止矣
 - Simplified: 不闻不若闻之，闻之不若见之，见之不若知之，知之不若行之；学至于行之而止矣
 - From [Xun Zi](#) (荀子 8.儒效 23) .

- Authors overview
 - Found **368 unique author entries**.
 - Problem: *entity fragmentation*. For example:
 - “Albert Einstein”
 - “Misattributed to Albert Einstein”
 - “Disputed with Albert Einstein”

These appeared as separate authors, though they should all map back to Einstein with different statuses.

- Most and least frequent authors
 - Top categories:
 - *unknown* (**5342 quotes**)
 - *Last words* (**1051**)
 - *English proverbs* (**897**)
 - Individuals like *George W. Bush* (674), *Winston Churchill* (504), *Emerson* (486), *Russell* (480), *MLK Jr.* (460).
 - Rare authors: many “Misattributed” or “Disputed” cases with only **1 quote each**.
- Heading context
 - Discovered **1724 unique heading values**, including decades (1930s), books (*Sidelights on Relativity*), and interviews.
 - *These were clean and useful as context metadata*.
- Quote and source lengths
 - Quotes ranged from **1 to 14,814 characters**. Sources ranged from **1 to 4356 characters**.
 - 1 length quotes were punctuations, which missed the hyperlink quotes.
 - Extremely long quotes were mostly conversations, speeches, interviews and even court debates.
 - Quotes <5 characters: **44 records**.
 - Sources <5 characters: **7336 records**.
 - Found **777 rows** with very short sources (<2 chars) and no heading context — many were useless (like “p. 11”, “Scene IV”) without heading context.
 - 99th percentile lengths: quotes ≈ **1784**, sources ≈ **751**. Anything beyond this was extreme.
- Special source patterns
 - ~**2.11% of sources** started with “translation/transliteration”.
 - That means the quote was another language (French, Chinese) etc. and the English translation was extracted in the source along with actual source.
 - Found **47 rows** with {{cite book}}. Some were meaningful references, but those ending with incomplete templates were useless. Because the extractor was written to avoid the hyperlinks to other pages.
 - Identified multiple trivial sources (e.g., “Couplet”, “Scene I”, “Chapter 3”, “Ibid.”).

- Duplicates
 - Found **88 duplicate records** on (quote, source).

4.2.2 Preprocessing and Cleaning

Based on the insights from EDA, I carried out the following fixes:

- Fixing Chinese proverb rows
 - Used a parser (parse_kv) to extract values like *Translation*, *Meaning*, *Pinyin* from the source text.
 - Where the quote was NaN or only punctuation, I inserted the extracted translation into the quote.
 - This rescued almost all 61 NaN and punctuation only Chinese quotes.
- Dropping invalid Wikiquote admin rows
 - Removed authors like “Wikiquote : Templates”, “Wikiquote : Utilities”, and “List of categories” as they were not quotes, just templates.
- Handling only-punctuation quotes
 - Dropped **32 rows** where the quote was only special characters and was not recoverable from the source.
- Short meaningless sources
 - Dropped rows where the source length <5 **and** heading context was NaN .
 - For cases where the quote contained :* (continuation markers), I split the text and moved the tail into the source (46 rows corrected this way).
- Filtering bad sources
 - Marked sources as meaningless if they matched patterns like “Scene IV”, “p. 11”, “Ibid.”, or {{cite...}}.
 - Candidates with no author + no heading_context + bad source = **0 rows** (none required removal).
 - Checked {{cite book}}: found **0 rows** where source was only a cite-book template (after parsing).
- Deduplication
 - Removed **88 duplicate rows** on (quote, source, heading_context).
- Author enrichment
 - Joined with extracted_quotes_about.csv to fill missing authors after re-extraction with better approach written for extracting most authors.
 - Updated **5288 rows** with improved author values.
- Trimming and normalization
 - Trimmed leading/trailing punctuation and whitespace from quotes and sources.
 - Standardized spacing and capitalization.
- Author status mapping
 - Split author into two fields:
 - status → canonical / about / misattributed / disputed.
 - target_name → clean person name.
 - This was necessary for the graph database.
 - Now, I could create a name only node for authors.
 - Saved this expanded version as quotes_spread.csv.

- Length control
 - Dropped **1813 quotes longer than 1000 chars**.
 - Truncated sources longer than 1000 chars.
 - Final dataset size: **27,799 records**.
- Output produced
 - quotes_spread.csv → with status mapping.
 - quotes_database_final.csv → final, ready for Neo4j.

The result was a clean, consistent dataset of 27,799 high-quality records.

5. Knowledge Graph:

After preprocessing the dataset, the next step was to load it into a database. For this, I used Neo4j, a graph database. Graphs make it natural to connect quotes to people and to handle cases like misattributions or disputed authorship. This way, when the assistant answers “who said this” or “is this disputed,” it is just traversing the graph.

5.1 Graph Schema:

Below is the simple Graph schema I used. There are two main node types:

- **Quote nodes:** It carries the quote and metadata. Each node stores:
 - **id:** a composite identifier combining status, person, and a quote fragment.
 - **text:** the actual quote.
 - **source:** extracted source
 - **heading_context:** the sub heading section (like *1930s*).
 - **status:** one of canonical, about, misattributed, or disputed.

Example:

```
(:Quote {
  id: "canonical|albert einstein|we shall, therefore, assume the complete physical equivalence of a gravitational field and a corresponding acceleration of the reference system",
  text: "We shall, therefore, assume the complete physical equivalence of a gravitational field and a corresponding acceleration of the reference system",
  source: "Statement of the equivalence principle in Yearbook of Radioactivity and Electronics (1907)",
  heading_context: "1900s",
  status: "canonical"
})
```

- **Person nodes:** It represents the people (target feature):

Example:

```
(:Person {
  name: "Albert Einstein",
  id: "albert-einstein"
})
```

- **Relationships:**

Attribution is modeled as relationships between quotes and people:

- **SAID_BY** → quote is said by the person (canonical).
- **ABOUT** → quote is about the person.
- **MISATTRIBUTED_TO** → quote is wrongly credited to the person.
- **DISPUTED_WITH** → quote is in dispute.

The same information also exists in the status property of each Quote node, but the relationships are what really matter in the graph. These will be used by assistant while answering queries.

- **Database Setup:**

The graph is deployed in Neo4j 5.26.9, in an instance called neo4j. After importing the final cleaned dataset, the database contained:

- **27,811 Nodes**(Quotes + People)
- **212 Person nodes**
- **27,799 relationships** in total

The relationships break down as:

- SAID_BY → 22,575
- ABOUT → 4,573
- MISATTRIBUTED_TO → 466
- DISPUTED_WITH → 185

There are no orphan Person nodes, meaning every Person node is referenced by at least one quote node.

- **Full-Text Index:**

To make retrieval fast and robust against partial fragments, I created a full-text index on the text property of Quote nodes, named quoteTextFT. It uses Lucene analyzers, which handle case insensitivity, tokenization, and scoring.

Example query:

```
CALL db.index.fulltext.queryNodes("quoteTextFT", "to be or not to be")
YIELD node, score
RETURN node.text, score
ORDER BY score DESC
LIMIT 5;
```

- **Retrieval Example**

I combine the full-text search with graph traversal so that both the quote and its relations are returned in one step.

Example query:

```
_CYPHER = """
CALL db.index.fulltext.queryNodes($index, $q) YIELD node, score
WITH node, score,
    [(node)-[r:SAID_BY|ABOUT|MISATTRIBUTED_TO|DISPUTED_WITH]->(p:Person)
    | {rel: type(r), name: p.name}] AS people
RETURN node.id AS id,
    node.text AS quote,
    node.source AS source,
    node.heading_context AS heading_context,
    node.status AS status,
    people,
    score
ORDER BY score DESC
LIMIT $limit
"""
```

This returns five best matching quotes with its properties, the connected people (with relation type), and the Lucene score. The assistant uses this structured result directly, so answers are deterministic and grounded in the graph.

6. Retrieval Module:

With the knowledge graph in place, the next piece was the retrieval module. Its job is to take a noisy or partial fragment from the ASR output and return the best matching quote node from Neo4j database, along with the metadata. This is where the full-text index and a reranking step come together.

- **Full-Text Querying**

The first stage is a call into Neo4j's full-text index (quoteTextFT) on the text property of Quote nodes. Instead of relying on one strict search, I generate several Lucene query variants for each fragment:

- **Exact phrase:** e.g., "to be or not to be"
- **Phrase with slop:** "to be or not to be"~3 (allows small gaps or order changes)
- **AND tokens:** to AND be AND not (forces all tokens to appear)
- **Wildcard tails:** to* be* not* (catches word stems)
- **Raw string:** the fragment itself, unmodified

This makes the retriever more robust to ASR noise, tokenization errors, or user input that doesn't match the stored quote texts word by word. Each query variant asks the index for up to **5 hits**, and the results are pooled.

- **Candidate Collection**

For each candidate, I also collect the connected people and their relationship types using an OPTIONAL MATCH over SAID_BY, ABOUT, MISATTRIBUTED_TO, and DISPUTED_WITH. This way the retriever always returns not just the quote text, but also structured metadata about who said it, whether it is about someone, or if it is disputed.

The result of this stage is a pool of candidate quotes, each with:

- id, text, source, heading_context, status
- people: a list of {rel, name} maps
- score: the Lucene relevance score

- **Reranking:**

Lucene scores alone are not always reliable, especially with short fragments. To address this, rerank candidates with a transparent scoring function.

- **Token coverage (55%):** fraction of query tokens that appear in the candidate quote.
 - **Normalized FT score (35%):** Lucene score capped and scaled.
 - **Phrase bonus (10%):** extra credit if the query tokens appear as a contiguous phrase.
- Each candidate is assigned a rerank score, and the list is sorted by this value.

- **Output:**

The retriever exposes two functions:

- search_topk(fragment, k=5) → returns the top-k candidates with their metadata and rerank scores.
- search_best(fragment) → it returns only the best candidate.

7. Language Understanding & Answering:

Once ASR provides text, three operations are performed:

1. extract the quote fragment,
2. decide the next action and what fields the user wants, and
3. answer strictly from graph facts. I keep deterministic rules up front and use the LLM only as a helper with JSON-only prompts.

7.1 Fragment extraction (LLM, JSON-only)

Task: pull only the quote words from a messy utterance and return them as JSON.

I call the LLM with temperature=0 and this exact system prompt:

```
# --- stricter extractor with examples ---
SYSTEM_EXTRACT = (
    "You are a precise span extractor. Return ONLY JSON: {\"fragment\": string}.\n"
    "Goal: extract ONLY the quote fragment to search for (not commands or author names).\n"
    "Remove command words: new quote, another quote, different quote, find, search, look up, \"\n"
    "complete, finish, continue, tell me, who said, source.\n"
    "Do NOT include author names or sources. Do NOT include trailing punctuation.\n"
    "If you cannot find at least 3 meaningful words of the quote, return {\"fragment\": \"\"}.\n"
    "\nExamples:\n"
    "Q: Find the quote: Albert Einstein was almost considered as a superhuman.\n"
    "A: {\"fragment\": \"Albert Einstein was almost considered as a superhuman\"}\n"
    "Q: Complete this new quote as an eminent pioneer in the realm of high.\n"
    "A: {\"fragment\": \"as an eminent pioneer in the realm of high\"}\n"
    "Q: Who said this? \"Two things are infinite...\n"
    "A: {\"fragment\": \"Two things are infinite\"}\n"
    "No extra text."
)
```

Post-processing: I parse the first {...} block, strip punctuation, remove any residual command words, and enforce ≥3 meaningful tokens. If it fails, the fragment is empty and I'll ask for "a few words from the quote."

7.2 Requested fields (Regex by default; LLM optional):

By default, I map the user's wording to fields with deterministic regex/keywords:

- "who said" → said_by
- "finish/complete it " → finish_quote
- "source/reference /where from" → source
- "disputed/misattributed " → disputed

For fuzzier phrasing, I can optionally enable a JSON-only LLM mapper (USE_LLM_INTENTS=1).

```
SYSTEM_REQUEST_FIELDS = (
    "You map a user question about a quote into a list of requested fields.\n"
    "Return ONLY JSON like: {\"fields\": [ ... ]}\n"
    "Valid fields:\n"
    " - \"said_by\"\n - \"about_person\"\n - \"misattributed_to\"\n - \"disputed_with\"\n - \"source\"\n - \"finish_quote\"\n - \"when\"\n"
    "Rules:\n1) Use plural intent if user asks for more than one thing.\n2) If unclear, return an empty list.\nNo extra text."
)
```

Examples:

- "who said this?" → { "fields": ["said_by"] }
- "finish it and give the source" → { "fields": ["finish_quote", "source"] }

I tried this approach but the speed was very very slow on my system so I had to drop this idea.

7.3 Intent hints & router (deterministic first; optional LLM)

I keep the router lightweight and deterministic. The dialogue layer handles quick intents ("new quote", reset, small talk) via regex, the main loop has guards to prevent chasing non-quote requests, and a follow-up rescue so short follow-ups ("who said it?") stick to the previous quote/session.

Deterministic intents cover: reset/clear, register/enroll & name parsing, set/list/test voice, rate/volume (words or numbers), "new/another quote", logout.

When I need fuzzier routing, I enable an **optional LLM router** with this exact JSON-only prompt:

```
SYSTEM_DECIDE = (  
    "Decide the next action for a quotes assistant.\n"  
    "Return ONLY JSON: {\"action\": one_of[\"answer_from_memory\", \"search_db\", \"clarify\", \"reset\"], \"query\": string}\n"  
    "Definitions:\n- answer_from_memory: user asks about the SAME quote and we already have context.\n"  
    "- search_db: user mentions a NEW quote or gives a fragment; provide a minimal search query.\n"  
    "- clarify: the request is unclear; ask the user to provide a quote fragment.\n"  
    "- reset: user asks to reset/clear/start over.\n"  
    "Rules:\n1) has_context=true means we already have the last quote facts.\n"  
    "2) If user says reset/clear/start over -> action=reset.\n"  
    "3) If has_context=true AND the user asks follow-ups (who/about/source/finish/disputed/when) possibly with 'this/that/the quote/it' -> action=answer_from_memory.\n"  
    "4) If has_context=false and the user asks a follow-up like 'who said this?' with no quote given -> action=clarify.\n"  
    "5) If the user provides a quote fragment or asks to find/complete a specific quote -> action=search_db with a minimal query (only the fragment words).\n"  
    "6) No extra text."  
)
```

If parsing fails, I default to search_db with the cleaned fragment.

7.4 Session & Speaker stickiness:

I tried to maintain per-user sessions in this project. If a Speaker ID is enabled, I only switch sessions when the user says logout/signout/bye etc or the new speech signal is long enough with a threshold similarity. For short clips, I keep a sticky active speaker briefly so rapid follow-ups feel natural. There is also a longer sticky window for follow-ups that contain “who/source/finish/disputed/when”.

7.5 Answering (deterministic; optional LLM for formatting)

The default answering path is **no hallucination**: I assemble strings directly from the retriever’s facts:

- **who** → names on SAID_BY (if none but MISATTRIBUTED_TO exists: “Unknown; often misattributed to ...”).
- **about** → names on ABOUT.
- **disputed** → list from DISPUTED_WITH and/or MISATTRIBUTED_TO.
- **source** → the source field.
- **finish** → only the quote text.
- **default** → full quote + author + source in one clean line.

If the user says *finish the quote*, I return the exact quote text (after light whitespace cleanup). For ambiguous “who/source” follow-ups with no fresh hit this turn, I answer from the last_facts in session.

8. Voice Interface:

This layer is where the assistant listens to the user, recognizes them, and replies in a way that feels consistent.

8.1 Input capture with VAD:

When I first tried capturing audio straight from the microphone, I quickly ran into the problem of long stretches of silence. I did not want the ASR to waste time on empty audio, so I placed a WebRTC VAD gate in the loop. I process the mic stream in 30 ms frames (480 samples at 16 kHz), and each frame is checked for speech. Recording begins when speech is detected, and it stops when there is 500 ms of silence. To keep things safe, I also added a 25 second maximum limit so the system never tries to handle overly long recordings.

I also added a small buffer so the first phonemes are not chopped off, and at the end I normalize the saved WAV to -3 dB so that downstream ASR and Speaker ID behave consistently. This small set of decisions made the system more manageable.

8.2 ASR with Whisper:

For transcription, I used **OpenAI Whisper model**. I wanted a balance between accuracy and responsiveness, so I keep two models side by side: the **small model** for normal turns where accuracy matters, and the **tiny model** as a fast path for very short commands or control intents. Both are lazy loaded, so that the startup is not slowed down.

I fixed the parameters to keep the output stable: temperature 0.0, no conditioning on previous text, no timestamps, and a no-speech threshold at 0.5. In practice, this means when I say a control word like “reset” it comes back instantly from the tiny model, while a longer fragment like a quote goes through the small model for better accuracy.

8.2 Speaker registration and identification

I built a Speaker ID module that stores embeddings of the users in a JSON file. To extract the user audio features from the audio, I used the pretrained speechbrain model. It converts the user speech into 192 dimensional embedding vectors. If something goes wrong, the module falls back to a lighter MFCC embedding (24 dimensions). Either way, each speaker can enroll through speaking multiple samples displayed on the terminal.

To register the user can say the below commands:

```
=== Voice Quotes Assistant ===
Press Enter to talk.
To register, say: 'register' (you'll read three short lines).
If you're already registered or don't want to register, just ask your quote question.
Examples: 'finish the quote two things are infinite' or 'who said this quote?'.
Say 'logout' or 'bye' to clear your session and use default TTS. Ctrl+C to exit.
```

The user is asked to speak 5 sentences one by one and the module extracts the embeddings of the recorded sample. Then the mean of the embeddings is saved with user name in the speakers.json file.

```
ENROLL_PROMPTS = [
    "The quick brown fox jumps over the lazy dog. Pack my box with five dozen liquor jugs.",
    "Common sense is a deposit of prejudices laid down before eighteen.",
    "Find me the quote Common sense is actually nothing more than a deposit of prejudices laid down in the mind prior to the age of eighteen.",
    "Has Albert Einstein said this quote, One of the sure signs of maturity is the ability to rise to the point of self criticism",
    "Numbers matter: three, five, seven, nine and forty two."
]
```

When a new utterance comes in, the system computes its embedding and compares it against the stored centroids with cosine similarity. If the best score is above a threshold (0.65) and the clip is at least 0.8 seconds, It accepts the match. If not, either stick with the current session's speaker or treat the speaker as unknown. This setup avoids annoying misclassifications while still feeling personal when it works.

8.3 TTS and per-user preferences:

I did not want a generic monotone voice; instead, I wanted each user to hear the assistant in the style they chose. For this, I used **pyttsx3** with the Windows SAPI5 driver, since it is offline, reliable and easy to implement. The two available voices on my Windows are:

- David
- Zira

And other variations are loudness, pitch, and rate user can change this using commands like set my rate to high. Increase my rate and volume.

```
You can also say: 'set my voice to Zira/David',  
'set my rate/speed to slow/medium/fast' or 'faster/slower',  
'set my volume to low/medium/high' or 'make it louder/quieter'.  
say 'test my voice' to hear your current voice.
```

The tricky part is that the engine's voice IDs are messy (long registry strings), while users just say “use Zira” or “speak faster.” To bridge this, I wrote a resolver that normalizes names, tries substring matches, and even does fuzzy matching. I implemented fuzzy matching so that even if ASR mishears voice names (e.g., ‘Zira’ as ‘Sira’), the assistant resolves correctly.

Preferences are stored in a JSON file (.cache/user_prefs.json). Each user has keys for voice, rate, and volume.

```
{  
  "default": {  
    "voice": "HKEY_LOCAL_MACHINE\\SOFTWARE\\Microsoft\\Speech\\Voices\\Tokens\\TTS_MS_EN-US_ZIRA_11.0"  
  },  
  "Sri": {  
    "voice": "HKEY_LOCAL_MACHINE\\SOFTWARE\\Microsoft\\Speech\\Voices\\Tokens\\TTS_MS_EN-US_DAVID_11.0"  
  },  
  "Samsung": {  
    "voice": "HKEY_LOCAL_MACHINE\\SOFTWARE\\Microsoft\\Speech\\Voices\\Tokens\\TTS_MS_EN-US_DAVID_11.0",  
    "rate": 150  
  }  
}
```

If a user says “faster” or “softer,” the system updates their prefs so the change persists across sessions. When I call `speak(text, user_id)`, it pulls those prefs, resolves the correct voice, and then speaks synchronously. This makes the assistant feel truly personalized.

9. Integration & Orchestration (main loop, sessions, sticky/rescue logic)

This chapter is about the main orchestration script, the session logic, and the sticky mechanisms to make the system coherent.

9.1 Design goal:

When I first wired all the pieces together, I realized that handling audio, ASR, retrieval, and TTS individually was easy enough but, the conversation fell apart if I didn't manage who was speaking and what context they were in. The orchestration layer is therefore designed to:

- Handle recording → ASR → intent handling → retrieval → answering → TTS as one continuous flow.
- Keep per-user sessions so that follow-ups like “who said it?” work.
- Prevent accidental speaker switches, while still letting people register and change voices naturally.
- Support deterministic control intents like reset, register, set voice or rate quickly

9.2 Dialogue loop:

File: main.py

The main loop (run_interactive) drives the whole system:

- Wait for the user to press Enter, then record with record_utterance_wav.
- Optionally run **Speaker ID** on the utterance. If a user is recognized, consider switching sessions. If not, apply sticky heuristics.
- Run **ASR** (Whisper model) to quickly detect control intents (reset, register, set voice). If a system intent is handled, the loop skips answering and goes back to listening.
- Otherwise run **full ASR** (Whisper small) to get the full transcript.
- If the utterance looks like a follow-up like “who said it?” etc. and there is no new recognized speaker, try a **long sticky rescue** from the previous session.
- Pass the transcript to the dialogue handler, which extracts fragments, queries the DB, and builds a factual answer.
- Call speak(reply, user_id) to speak back in the user's preferred voice.
- This end-to-end cycle gives the assistant its conversational rhythm: listen → understand → retrieve → speak.

9.3 Session management:

File: session.py

Sessions are stored in a simple in-memory dictionary keyed by user ID. Each session carries last_facts, which represent the most recent retrieved quote and metadata. This enables follow-ups like:

- User: “finish the quote two things are infinite” → system retrieves the full Einstein quote.
- User: “who said it?” → even if no new fragment is given, the system answers from last_facts in that session.

The functions are minimal:

- get_session(user_id) → returns or creates a session.
- clear_session(user_id) → clears last_facts for that user.
- clear_all_sessions() → resets all.

This simplicity was enough to get deterministic behavior from the model.

9.4 Sticky speakers and rescue logic

File: main.py

I did not want the system to jump between users just because of a short noise. So, I set up three levels of stickiness:

- **Short sticky (≤0.9s):** if the clip is very short and close in time to the last speaker, keep the same session.
- **Long sticky (≤90s, follow-ups only):** if a question looks like a follow-up (“who said”, “finish”), and the last speaker was recognized recently, continue with their session even if no ID match is detected this turn. Switching to a new speaker requires stricter conditions:
 - Utterance duration ≥1.2s.
 - Cosine score from Speaker ID ≥0.66.
 - Or, the user explicitly says “This is John / I am John” as a self-intro.

These rules balance stability with flexibility — the assistant doesn't flip identity too easily, but it does change when the cues are strong enough.

9.5 Intent handling:

Files: main.py, intent_mapper.py

Deterministic regexes cover the common control cases:

- Reset/clear context.
- Register/enroll with a name.
- Set/list/test voice.
- Adjust rate/volume (words or numbers).
- "new/another quote".

For fuzzier phrasing, I can enable the LLM intent mapper, which returns compact JSON {intent, slots, confidence}. This is guarded by the USE_LLM_INTENTS variable and falls back to a query if parsing fails.

```
{ "intent" : "set_rate", "slots" : { "rate": ... }, "confidence": "high" }
```

When USE_LLM_INTENTS is enabled, the project becomes very slow. So I kept this feature optional and only for demonstration purposes.

9.6 Dialogue handling:

File: dialogue.py

The dialogue handler takes the full transcript and session ID. It:

- Responds to small talk ("thanks") with a fixed ack.
- Clears context on "new quote".
- Calls the LLM extractor to pull a clean fragment from the utterance.
- Always searches the DB with the fragment, not the whole utterance.
- Saves the result in the session's last_facts and uses the LLM to build an answer from the facts.
- If no new fragment is found but there is context, it answers from memory.
- Otherwise it asks the user to give a few words from the quote.

This keeps the retrieval loop predictable: always fragment → search → answer.

9.7 Retrieval module:

File: retriever.py

The retriever runs multiple **full-text query variants** on the Neo4j index: exact phrase, phrase with slop, AND tokens, wildcard tails, and raw fragment. Candidates are pooled, deduplicated, and re-ranked with a simple transparent scorer. The best hit is returned as structured data with text, source, context, and people relations. This structured result is what the assistant uses to produce answers.

9.8 LLM helpers:

File: llm.py

I use the **Mistral 7B LLM model** only in narrow, constrained ways:

- **Fragment extraction:** pull only the quote span, never author names or commands.
- **Field mapping:** decide which fields the user is asking for (who/source/finish/disputed/etc.).
- **Answer formatting:** when multiple DB results exist, rephrase them into a short natural answer but still grounded in the DB.
- **Decision:** pick whether to search DB, answer from memory, ask to clarify, or reset.

This approach ensures the LLM is never an oracle: it only works within the facts already retrieved.

10. Evaluation:

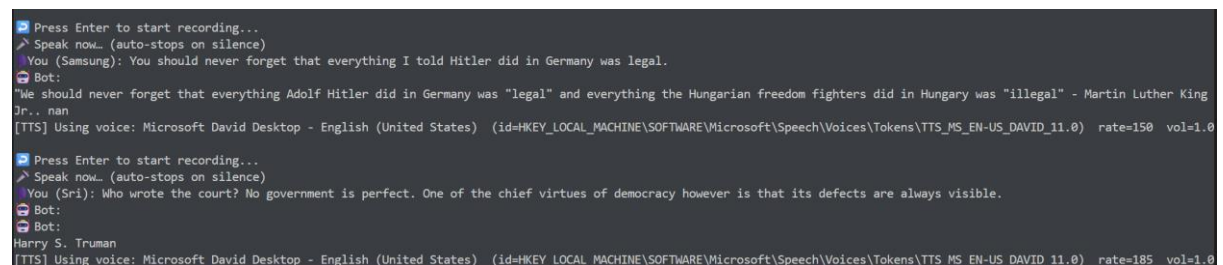
I focused my evaluation on the retrieval and answering layer, since ASR, Speaker ID, and TTS are preexisting modules whose accuracy has been established in prior work.

Retrieval accuracy. Using a test set of 95 fragments sampled from the database, I measured Top-1 accuracy, Top-5 recall, and MRR@5. The system achieved **93.7% Top-1 accuracy**, **96.8% Top-5 recall**, and **0.95 MRR@5**, with median retrieval latency of approx. 22 ms. This shows that the retriever plus simple reranker reliably brings the correct quote to the top results.

Speaker Recognition Accuracy: Due to a limited number of users (myself, one friend and google chatbot voice for testing), I could save just three users for this project. The chatbot recognises us most of the time correctly given, the length of the speech should be long enough and not mixed with another speech signal. Also, the recognition is affected if the user registers his speech in a different environment and logs in from different environment. Otherwise, it works fine.

Human evaluation: I interacted with it for typical scenarios: *finish the quote...*, *who said it?*, *what is the source?*, and *is it disputed?* I also tried requests such as “say it again” or “who wrote this”. In most cases the assistant:

- Successfully completed quotes and attributed them to the correct author.
- Correctly remembered the previous turn and answered follow-ups from context (“*who said this*” → *Isaac Asimov*).
- Repeated a retrieved quote when asked again, showing session stickiness.



```
Press Enter to start recording...
Speak now... (auto-stops on silence)
You (Samsung): You should never forget that everything I told Hitler did in Germany was legal.
Bot:
"We should never forget that everything Adolf Hitler did in Germany was "legal" and everything the Hungarian freedom fighters did in Hungary was "illegal" - Martin Luther King Jr.. nan
[TTS] Using voice: Microsoft David Desktop - English (United States) (id=HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Speech\Voices\Tokens\TTS_MS_EN-US_DAVID_11.0) rate=150 vol=1.0
Press Enter to start recording...
Speak now... (auto-stops on silence)
You (Sri): Who wrote the court? No government is perfect. One of the chief virtues of democracy however is that its defects are always visible.
Bot:
Harry S. Truman
[TTS] Using voice: Microsoft David Desktop - English (United States) (id=HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Speech\Voices\Tokens\TTS_MS_EN-US_DAVID_11.0) rate=185 vol=1.0
```

Limitations. Some topic-style queries (“*quote about Gandhi*”) still failed if the fragment was too short. Sources were occasionally noisy or missing (NaN). Temporal questions like “*when?*” were not consistently handled.

11. Future Work & Conclusion:

In this project, I deliberately kept most of the decision logic lightweight and **regex-based**. Only the fragment extraction step uses the LLM. I did experiment with letting the LLM decide all actions (intent classification, dialogue state, routing), but the latency on my local machine was too high to be practical. The result was that I could not iterate or test in real time.

By contrast, the regex approach was deterministic and instant, which let me build and evaluate the system end-to-end. The trade-off is that regex rules are brittle and must be enumerated by hand.

Looking ahead, as computing resources improve, I would like to revisit a **fully LLM-driven decision layer**. Modern large models are already strong at **zero-shot intent detection and decision making**, so an LLM-based router could remove the need for handcrafted patterns and allow more natural, flexible conversation. The assistant could accept fuzzier phrasing, handle edge cases gracefully, and generalize to new actions without explicit rules.

In conclusion, this project demonstrates that it is possible to combine a knowledge graph, retrieval, and lightweight dialogue logic into a working **voice-first assistant**. The system is accurate in retrieval, grounded in database facts, and capable of simple multi-turn interaction. The evaluation shows both its promise and its limits. With regex-based decision rules, it is responsive enough to run on modest hardware, but the long-term potential lies in migrating more of the orchestration to modern LLMs once the computational constraints are lifted.